Generative AI Mastery Guide — Senior Architect (Python)

> Core Gen AI topics and techniques (not traditional ML) for a senior architect working in Python.

---

1. LLM Foundations (Architect-Level Understanding)

- **Transformer Architecture** — self-attention, multi-head attention, positional encoding, encoder-decoder vs decoder-only
- **Tokenization** — BPE (Byte Pair Encoding), SentencePiece, tiktoken; token limits, cost implications, multilingual token efficiency
- **Context Windows** — fixed vs. sliding window, long-context models (128K+), context window management strategies
- **Inference Mechanics** — autoregressive decoding, temperature, top-k, top-p (nucleus sampling), repetition penalty, stop sequences
- **Model Families** — GPT-4/4o, Claude (Opus/Sonnet/Haiku), Gemini, Llama 3, Mistral, Mixtral, Command R, Phi, Qwen — know strengths, trade-offs, and licensing
- **Scaling Laws** — Chinchilla scaling, compute-optimal training, emergent abilities

---

2. Prompt Engineering (Advanced)

- **Core Techniques** — zero-shot, few-shot, chain-of-thought (CoT), self-consistency, tree-of-thought
- **System Prompt Design** — persona setting, constraints, output formatting, guardrails
- **Structured Output** — JSON mode, XML tagging, function-calling schemas, Pydantic model enforcement
- **Prompt Chaining** — decomposing complex tasks into sequential LLM calls
- **Meta-Prompting** — prompts that generate or refine prompts
- **Prompt Templating** — Jinja2-based templates, parameterized prompts, version control for prompts
- **Prompt Injection Defense** — input sanitization, delimiter strategies, instruction hierarchy

Python Libraries: `langchain`, `guidance`, `instructor`, `outlines`, `jinja2`

---

3. Retrieval-Augmented Generation (RAG)

- **Core RAG Pipeline** — ingest → chunk → embed → index → retrieve → augment → generate
- **Document Chunking Strategies** — fixed-size, recursive, semantic chunking, parent-child chunking, late chunking
- **Embedding Models** — OpenAI `text-embedding-3`, Cohere Embed v3, `sentence-transformers`, BGE, E5, Jina
- **Vector Databases** — Pinecone, Weaviate, Qdrant, Milvus, ChromaDB, pgvector, FAISS
- **Retrieval Strategies** — dense retrieval, sparse retrieval (BM25), hybrid search, re-ranking (Cohere Rerank, cross-encoders)
- **Advanced RAG Patterns** — multi-query retrieval, HyDE (Hypothetical Document Embeddings), self-RAG, corrective RAG, agentic RAG, GraphRAG
- **Multimodal RAG** — embedding and retrieving images, tables, PDFs
- **Evaluation** — retrieval precision/recall, faithfulness, answer relevance (RAGAS framework)

Python Libraries: `langchain`, `llama-index`, `chromadb`, `qdrant-client`, `pinecone`, `ragas`, `unstructured`

---

4. AI Agents & Tool Use

- **Agent Architectures** — ReAct, Plan-and-Execute, LATS, Reflexion
- **Tool/Function Calling** — defining tool schemas, OpenAI function calling, Anthropic tool use, parallel tool calls
- **Frameworks** — LangGraph, CrewAI, AutoGen, Smolagents, OpenAI Assistants API, Anthropic Claude agents
- **Multi-Agent Systems** — orchestrator-worker, debate, delegation, hierarchical agents
- **Memory for Agents** — short-term (conversation buffer), long-term (vector store), episodic, entity memory
- **Human-in-the-Loop** — approval gates, escalation, interactive agent flows
- **Code Execution Agents** — sandboxed code interpreters, E2B, Modal
- **Browser/Computer Use Agents** — Playwright-based, Anthropic computer use, web scraping agents

Python Libraries: `langgraph`, `crewai`, `autogen`, `smolagents`, `e2b`

---

5. Fine-Tuning & Model Customization

- **Parameter-Efficient Fine-Tuning (PEFT)** — LoRA, QLoRA, DoRA, Adapters, Prefix Tuning
- **Full Fine-Tuning** — when and why, hardware requirements, distributed training
- **Instruction Tuning** — dataset formats (Alpaca, ShareGPT, ChatML), data quality > quantity
- **RLHF / RLAIF** — reward modeling, PPO, DPO (Direct Preference Optimization), KTO
- **Synthetic Data Generation** — using LLMs to generate training data, self-instruct, Evol-Instruct
- **Continued Pre-Training** — domain adaptation on custom corpora
- **Fine-Tuning APIs** — OpenAI fine-tuning, Anthropic fine-tuning, Together AI, Anyscale

Python Libraries: `transformers`, `peft`, `trl`, `datasets`, `axolotl`, `unsloth`

---

6. LLM Orchestration & Application Frameworks

- **LangChain** — chains, LCEL (LangChain Expression Language), runnables, callbacks
- **LlamaIndex** — data connectors, index types, query engines, routers
- **LangGraph** — stateful graphs, conditional edges, cycles, persistence, streaming
- **Semantic Kernel** — Microsoft's orchestration SDK (Python support)
- **Haystack** — pipeline-based NLP/Gen AI framework
- **Choosing the Right Framework** — when to use frameworks vs. raw API calls, avoiding over-abstraction

---

7. Evaluation, Testing & Observability

- **LLM Evaluation Metrics** — BLEU, ROUGE, BERTScore (reference-based); G-Eval, LLM-as-Judge (reference-free)
- **RAG Evaluation** — RAGAS (faithfulness, answer relevance, context precision/recall)
- **Agent Evaluation** — task completion rate, tool selection accuracy, trajectory analysis
- **Red Teaming** — adversarial testing, jailbreak attempts, edge case discovery
- **Regression Testing** — prompt regression suites, golden datasets, CI/CD for prompts

- **Observability & Tracing** — LangSmith, LangFuse, Arize Phoenix, OpenLLMetry, Weights & Biases
- **Cost & Latency Monitoring** — token usage tracking, latency percentiles, budget alerting

Python Libraries: `ragas`, `deepeval`, `promptfoo`, `langsmith`, `langfuse`, `opentelemetry`

---

8. Serving, Deployment & Infrastructure

- **Inference Engines** — vLLM, TGI (Text Generation Inference), TensorRT-LLM, Ollama, llama.cpp
- **Optimization Techniques** — KV-cache, continuous batching, speculative decoding, PagedAttention
- **Quantization** — GPTQ, AWQ, GGUF, bitsandbytes (4-bit, 8-bit), when to quantize
- **API Gateway Patterns** — load balancing across providers, fallback chains, rate limiting, retry with exponential backoff
- **Model Routing** — routing to different models by complexity/cost (small model first, escalate to large)
- **Streaming** — SSE (Server-Sent Events), streaming token-by-token, WebSocket patterns
- **Containerization** — Docker for LLM services, GPU-aware containers, Kubernetes for scaling
- **Serverless / Managed** — AWS Bedrock, Azure OpenAI, GCP Vertex AI, Fireworks, Together, Groq, Replicate

Python Libraries: `vllm`, `litellm`, `openai`, `anthropic`, `boto3` (Bedrock), `fastapi`

---

9. Guardrails, Safety & Governance

- **Content Filtering** — input/output moderation, PII detection and redaction, toxicity detection
- **Prompt Injection Mitigation** — input validation, canary tokens, dual-LLM patterns, instruction hierarchy
- **Hallucination Mitigation** — grounding with RAG, citation enforcement, confidence scoring, abstention strategies
- **Guardrail Frameworks** — Guardrails AI, NeMo Guardrails, LLM Guard

- **Responsible AI** — bias detection, fairness testing, transparency/explainability for Gen AI outputs
- **Data Privacy** — PII handling, data residency, opt-out / consent, EU AI Act awareness
- **Model Governance** — model cards, versioning, audit trails, A/B testing for prompts/models

Python Libraries: `guardrails-ai`, `nemo-guardrails`, `presidio`, `llm-guard`

---

10. Multimodal Generative AI

- **Vision-Language Models** — GPT-4V/4o, Claude Vision, Gemini Pro Vision, LLaVA — image understanding, chart/diagram interpretation
- **Image Generation** — Stable Diffusion, DALL·E 3, Midjourney concepts; `diffusers` library, ControlNet, img2img, inpainting
- **Audio/Speech** — Whisper (STT), TTS (ElevenLabs, OpenAI TTS), voice cloning, real-time voice agents
- **Video Generation** — Sora, Runway, Kling — emerging architectures
- **Multimodal Embeddings** — CLIP, SigLIP, embedding images + text in the same space
- **Multimodal RAG** — extracting and retrieving across text, images, tables, and audio

Python Libraries: `diffusers`, `transformers`, `openai`, `whisper`, `clip`

---

11. Architecture & Design Patterns for Gen AI Systems

- **Architectural Patterns**:
  - Gateway Pattern — single entry point for multiple LLM providers
  - Semantic Cache — cache by meaning, not exact query (use embeddings)
  - Routing Pattern — classify input → route to specialized model/chain
  - Fallback Chain — primary model → fallback model → graceful degradation
  - MapReduce for LLMs — split large docs → parallel summarize → combine
  - Iterative Refinement — generate → critique → regenerate loop
- **Design Considerations**:
  - Idempotency and non-determinism handling
  - Async-first design (LLM calls are I/O-bound)
  - Circuit breaker for API calls

    - Token budget management per request
    - Caching strategies (exact match, semantic, KV cache)
- **Data Architecture**:
    - Embedding pipeline design (batch vs. real-time)
    - Vector DB schema design and indexing strategies
    - Hybrid storage (relational + vector + document)
    - Data freshness and re-indexing strategies

---

12. Advanced & Emerging Topics

- **MCP (Model Context Protocol)** — Anthropic's standard for tool/service integration; building MCP servers and clients
- **Compound AI Systems** — combining multiple models, retrievers, and tools into coherent systems
- **Knowledge Graphs + LLMs** — GraphRAG, entity extraction, ontology-grounded generation
- **Long-Context Techniques** — context compression, recursive summarization, memory-augmented transformers
- **Structured Generation** — constrained decoding (Outlines, Guidance), guaranteed JSON/schema compliance
- **Distillation** — training smaller models from larger model outputs
- **Mixture of Experts (MoE)** — Mixtral architecture, sparse routing, efficiency gains
- **Local/Edge LLMs** — running models on-device (Ollama, llama.cpp, MLX for Apple Silicon)
- **LLMOps** — prompt versioning, experiment tracking, model registry, CI/CD pipelines for AI applications

---

13. Essential Python Libraries — Quick Reference

Category	Libraries
LLM APIs	`openai`, `anthropic`, `google-generativeai`, `litellm`, `boto3`
Orchestration	`langchain`, `llama-index`, `langgraph`, `haystack`

Category	Libraries
Agents	`langgraph`, `crewai`, `autogen`, `smolagents`
Embeddings & Vectors	`sentence-transformers`, `chromadb`, `pinecone`, `qdrant-client`, `faiss`
Fine-Tuning	`transformers`, `peft`, `trl`, `datasets`, `unsloth`, `axolotl`
Structured Output	`instructor`, `outlines`, `guidance`, `pydantic`
Serving	`vllm`, `fastapi`, `uvicorn`, `litellm`
Evaluation	`ragas`, `deepeval`, `langsmith`, `langfuse`
Guardrails	`guardrails-ai`, `nemo-guardrails`, `presidio`
Multimodal	`diffusers`, `whisper`, `transformers`, `clip`
Document Processing	`unstructured`, `pymupdf`, `docling`, `marker`

14. Suggested Learning Path (Senior Architect)

Phase 1 — Foundations (Weeks 1–3): Transformer internals, tokenization, prompt engineering, structured outputs, calling LLM APIs directly

Phase 2 — RAG Mastery (Weeks 4–6): End-to-end RAG pipeline, chunking strategies, vector DBs, hybrid search, evaluation with RAGAS

Phase 3 — Agents & Orchestration (Weeks 7–9): Function calling, LangGraph agents, multi-agent patterns, memory, human-in-the-loop

Phase 4 — Production (Weeks 10–12): Serving (vLLM), guardrails, observability, cost optimization, architectural patterns, CI/CD for AI apps

Phase 5 — Advanced (Ongoing): Fine-tuning (LoRA/QLoRA), multimodal, MCP, knowledge graphs, edge deployment, emerging research